

RSA encryption, Shor factoring algorithm, and quantum Fourier transform

Peter W. Shor, Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer, SIAM J. Comput. **26**, 1484 (1997), arXiv:quant-ph/9508027 main

$$299 = \underline{13} \times \underline{23}$$

very important point

Dr. Arun Sehrawat

video



Modular (remainder) arithmetic and multiplicative group of co-primes

$$x \equiv y \pmod{N}$$

- $y \equiv \underbrace{x \pmod{N}}_{\text{computing remainder}} \leftrightarrow \underbrace{x}_{\text{dividend}} = \underbrace{y}_{\text{remainder}} + \underbrace{k}_{\text{quotient}} \times \underbrace{N}_{\text{divisor}} \leftrightarrow x \text{ and } y \text{ are equal up to a multiple of } N.$

$$13\text{hr} \equiv 1\text{pm}$$

$$N = 12 \quad \sqrt{13 = x} \quad 1 = k$$

$$\frac{12}{1} = y$$

$$13 \equiv 1 \pmod{12}$$

$$\mathbb{Z}_{12} = \{0, 1, 2, \dots, 11\} \text{ additive gp}$$

$$6 + 7 = 13 \equiv 1 \pmod{12}$$

$$\textcircled{3} (a+b)+c = a+(b+c)$$

$$\textcircled{1} 6 + 0 = 6$$

$$\textcircled{2} 6 + 6 = 12 \equiv 0 \pmod{12} \quad \text{Prime: } 2, 3, 5, 7, 11, 13 \dots$$

- E and M are called co-primes, if they have no common factors, that is $\gcd(E, M) = 1$.

$$6 = 2 \times 3 \quad \gcd(6, 55) = 1$$

$$55 = 5 \times 11$$

$$3 = 1 \times 3$$

$$5 = 1 \times 5$$

- $\mathbb{Z}_M^* = \{E \mid \underbrace{0 \leq E \leq M-1}_{\mathbb{Z}_M} \text{ and } \gcd(E, M) = 1\} = \text{the multiplicative group of integers modulo-}M.$

$$\gcd(3, 5) = 1$$

$$\mathbb{Z}_{264}^* = \{1, 5, 7, 13, 17, 19, 23, 25, 29, 31, 35, 37, 41, 43, 47, 49, 53, 59, 61, 65, 67, 71, 73, 79, 83, 85, 89, 91, 95, 97, 101, 103, 107, 109, 113, 115, 119, 125, 127, 131, 133, 137, 139, 145, 149, 151, 155, 157, 161, 163, 167, 169, 173, 175, 179, 181, 185, 191, 193, 197, 199, 203, 205, 211, 215, 217, 221, 223, 227, 229, 233, 235, 239, 241, 245, 247, 251, 257, 259, 263\}$$

$$\gcd(2, 264) = 2$$

$$\gcd(3, 264) = 3$$

$$5 \times 7 = 35, \quad 5 \times 53 = 265 = 1 + 264, \quad 19 \times 31 = 589 = 61 + 2 \times 264$$

$$\equiv 35 \pmod{264}$$

$$\equiv 1 \pmod{264}$$

$$\equiv 61 \pmod{264}$$

$$5 \times 1 \equiv 5$$

$\gcd(0, M) = M$, " $\equiv$ " means congruent

N. D. Mermin, *Quantum Computer Science: An Introduction*, Chapt 3.

M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Appendix 4 and Chapt 5.

RSA (Rivest–Shamir–Adleman) encryption

Alice -----> Bob

Step 1: Bob "randomly" chooses two ^{odd} distinct prime numbers p and q , say, each of 1000 digits.

Step 2: Bob computes $N = pq$ and $M = (p-1)(q-1)$. Without knowing p and q , M cannot be calculated.

Step 3: Bob chooses an **encoding** integer $E \in \mathbb{Z}_M^*$.

Step 4: Bob sends N and E to Alice through a public channel.



Step 5: Alice encodes her message $a < N$ as $b \equiv a^E \pmod{N}$ and then sends b to Bob through a public channel.

Step 6: Bob computes **decoding** number (multiplicative inverse) D such that $1 \equiv DE \pmod{M}$, and then decode the original message $a \equiv b^D \pmod{N}$. Without knowing M , D cannot be calculated.

best classical algo $\sim 2^{O(n^{1/3})}$ steps

Shor algo on QC $\sim O(n^3)$ steps

$$n = \lceil \log_2 N \rceil = \text{min number of bits needed to rep. } N$$

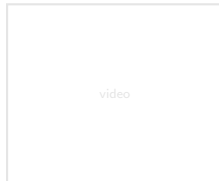
$$\lceil 3.2 \rceil = 4$$

$$N = 7 \equiv 111$$

Finding p and q from N is called prime factorization, which is practical impossibility for a "classical" computer for a 2000-digit number.

$$p \sim 10^{1000}$$

$$N \sim 10^{2000}$$



RSA (Rivest–Shamir–Adleman) encryption: Example

Alice -----> Bob

Step 1: Bob randomly chooses two distinct ^{odd} prime numbers $p = 13$ and $q = 23$.

Primality test
 Yes
 No

Step 2: Bob computes $N = pq = 299$ and $M = (p-1)(q-1) = 264$.

12×22

Step 3: Bob chooses an **encoding** integer $E = 5 \in \mathbb{Z}_M^*$ =

{1,5,7,13,17,19,23,25,29,31,35,37,41,43,47,49,53,59,61,65,67,71,73,79,83,85,89,91,95,97,101,103,107,109,113,115,119,125,127,131,133,137,139,145,149,151,155,157,161,163,167,169,173,175,179,181,185,191,193,197,199,203,205,211,215,217,221,223,227,229,233,235,239,241,245,247,251,257,259,263}

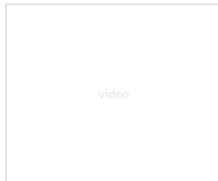
Step 4: Bob sends $N = 299$ and $E = 5$ to Alice through a public channel.

$$20^5 = 3200000 = 10702 \times 299 + 102$$

Step 5: Alice encodes her message $a = 20$ as $b = a^E \pmod{N} = 102$ and then sends b to Bob through a public channel.

Step 6: Bob computes **decoding** number (multiplicative inverse) $D = 53$ such that $1 \equiv DE \pmod{M}$, and then decode the original message $b^D \pmod{N} = 20$.

$$299 \overline{) 102^{53}}$$



Basis of RSA encryption: Fermat's little theorem

- $y \equiv x \pmod{p} \Leftrightarrow x = y + kp$

- Fermat's little theorem:** If p is a prime number and an integer a is **not** divided by p , then $1 \equiv a^{p-1} \pmod{p}$.

For example, $p = 13$, $\mathbb{Z}_p^* = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$, $3^{13-1} = 531441 = 1 + 40880 \times 13$

$$2^2 = 4, \quad 2^3 = 8, \quad 2^4 = 16 = 13 + 3 \equiv 3 \pmod{13}, \quad 2^5 = 32 = 2 \times 16 \equiv 6 \pmod{13}, \quad 2^6 = 64 = 12 \times 5 + 4 \equiv 4 \pmod{13}, \quad 2^7 = 128 = 10 \times 13 + 8 \equiv 8 \pmod{13}, \quad 2^8 = 256 = 19 \times 13 + 9 \equiv 9 \pmod{13}, \quad 2^9 = 512 = 39 \times 13 + 5 \equiv 5 \pmod{13}, \quad 2^{10} = 1024 = 78 \times 13 + 10 \equiv 10 \pmod{13}, \quad 2^{11} = 2048 = 157 \times 13 + 7 \equiv 7 \pmod{13}, \quad 2^{12} = 4096 = 315 \times 13 + 14 \equiv 14 \pmod{13} \equiv 1 \pmod{13}$$

- As a result of Fermat's little theorem, we have

Similarly, for distinct prime q

$$\begin{cases} a \equiv a^{1+k(p-1)(q-1)} \pmod{p} \\ a \equiv a^{1+k(p-1)(q-1)} \pmod{q} \end{cases}$$

- $1 \equiv \text{DE} \pmod{(p-1)(q-1)} \Leftrightarrow \text{DE} = 1 + k(p-1)(q-1)$

- So, we have $a \equiv a^{\text{DE}} \pmod{pq}$

Encoding: $a \rightarrow a^{\text{DE}} \pmod{pq}$
Decoding: $a^{\text{DE}} \rightarrow a \pmod{pq}$

$$1 \equiv 2^{b-1} \pmod{b}$$

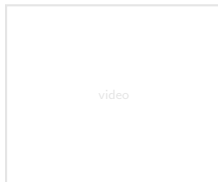
$$1 = \binom{k(q-1)}{1} \equiv \left(2^{b-1}\right)^{k(q-1)} \pmod{b}$$

$$1 \equiv a^{k(b-1)(q-1)} \pmod{b}$$

$$a \equiv a^{1+k(p-1)(q-1)} \pmod{p}$$

Number of elements in (order of the group) $\mathbb{Z}_N^*$ are M , and $1 \equiv a^M \pmod{N}$ Euler's theorem.

N. D. Mermin, *Quantum Computer Science: An Introduction*, Chapt 3.



Prime factorization

What are the prime factors of $N = 299$?

Step 1: find $x \pmod N \not\equiv \pm 1$ but $x^2 \pmod N \equiv 1$

$$\begin{aligned} x^2 &= 1 + kN \\ x^2 - 1 &= kN \end{aligned}$$

$$(x-1)(x+1) = k \times N = k \times p \times q$$

For $x = -1, \dots, 300$,

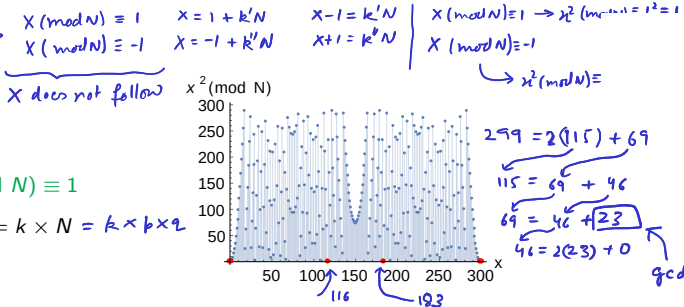
$$\{x, x^2 \pmod N\} = \{\{-1, 1\}, \{0, 0\}, \{1, 1\}, \{2, 4\}, \{3, 9\}, \{4, 16\}, \{5, 25\}, \{6, 36\}, \{7, 49\}, \{8, 64\}, \{9, 81\}, \{10, 100\}, \\ \{11, 121\}, \{12, 144\}, \{13, 169\}, \{14, 196\}, \{15, 225\}, \{16, 256\}, \{17, 289\}, \dots, \{111, 62\}, \{112, 285\}, \\ \{113, 211\}, \{114, 139\}, \{115, 69\}, \{116, 1\}, \{117, 234\}, \{118, 170\}, \{119, 108\}, \{120, 48\}, \{121, 289\}, \dots, \{177, 233\}, \\ \{178, 289\}, \{179, 48\}, \{180, 108\}, \{181, 170\}, \{182, 234\}, \{183, 1\}, \{184, 69\}, \{185, 139\}, \{186, 211\}, \dots, \{294, 25\}, \\ \{295, 16\}, \{296, 9\}, \{297, 4\}, \{298, 1\}, \{299, 0\}, \{300, 1\}\}$$

$x = 116$

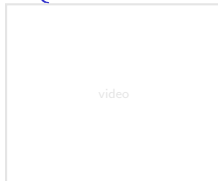
Step 2: $p = \gcd(x-1, N) = \gcd(115, 299) = 23$
 $q = \gcd(x+1, N) = \gcd(117, 299) = 13$

$$\frac{x-1}{N} = \frac{5}{13}, \quad \frac{x+1}{N} = \frac{9}{23}$$

Euclid's algorithm is an efficient way to compute the greatest common divisor.



$$\begin{aligned} (116)^2 \pmod{299} &\equiv 1 \\ 116 \pmod{299} &\not\equiv \pm 1 \end{aligned}$$



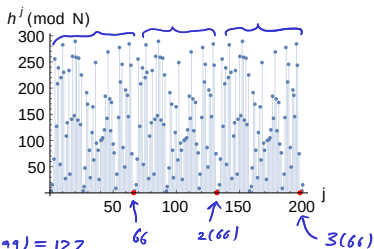
Prime factorization is equivalent to order finding (1)

(p, 2)

Step 1: How to find $x \pmod{N} \not\equiv \pm 1$ but $x^2 \pmod{N} \equiv 1$?

Pick a random integer $h < N$ and compute $h^j \pmod{N}$ for $j = 1, \dots$ till you find **smallest** $j = r$ such that $h^r \pmod{N} \equiv 1$. That r will be the **order**. If r is **even**, then $x = h^{(r/2)}$. The probability of r being even and $\underline{h^{(r/2)} \pmod{N} \not\equiv -1}$ is at least $\frac{1}{2}$.

$$h^{(r/2)} \pmod{N} \not\equiv 1$$



For $h = 4 < 299 = N$ and $j = 1, \dots, 200$,

$\{j, h^j \pmod{N}\} = \{\{1, 4\}, \{2, 16\}, \{3, 64\}, \{4, 256\}, \{5, 127\}, \{6, 209\}, \{7, 238\}, \dots, \{61, 186\}, \{62, 146\}, \{63, 285\}, \{64, 243\}, \{65, 75\}, \{66, 1\}, \{67, 4\}, \{68, 16\}, \{69, 64\}, \{70, 256\}, \dots, \{130, 243\}, \{131, 75\}, \{132, 1\}, \{133, 4\}, \{134, 16\}, \{135, 64\}, \dots, \{195, 285\}, \{196, 243\}, \{197, 75\}, \{198, 1\}, \{199, 4\}, \{200, 16\}\}$

Quantum computer is used to comp $h^j \pmod{N}$

$$r = 66, x = h^{(r/2)} = 4^{33} = 73786976294838206464$$

$$4^{66} \pmod{299} \equiv 1$$

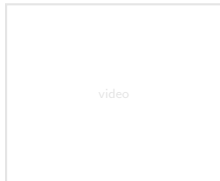
$$4^{33} \pmod{299} \not\equiv -1 \checkmark$$

Step 2: $p = \gcd(x - 1, N) = \gcd(73786976294838206463, 299) = 23$
 $q = \gcd(x + 1, N) = \gcd(73786976294838206465, 299) = 13$

When it does not work: $3^{33} \pmod{299} \equiv 1$, $10^{66} \pmod{299} \equiv 1$ and $10^{33} \pmod{299} \equiv -1 \equiv 298$

When $h = 3$

$h = 10$



Quantum algorithm for order-finding

$$h=4 \quad f(0)=1 \rightarrow |1\rangle \equiv |0000\rangle$$

$$f(1)=4 \rightarrow |4\rangle \equiv |0010\rangle$$

$$f(2)=16 \rightarrow |16\rangle \equiv |1000\rangle$$

Cyclic permutation: $U|f(l)\rangle = |f(l+1)\rangle$ where $f(l+r) = f(l) := h^l \pmod N$ and $l = 0, \dots, r-1$.

$$|0\rangle^{\otimes m} = |0\rangle|0\rangle|0\rangle\dots|0\rangle \equiv |00\dots0\rangle$$

$$\begin{pmatrix} |u_0\rangle \\ |u_1\rangle \\ \vdots \\ |u_{r-1}\rangle \end{pmatrix} = \frac{1}{\sqrt{r}} \underbrace{\begin{pmatrix} 1 & 1 & \dots & 1 \\ 1 & \omega^{-1} & \dots & \omega^{-(r-1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-(r-1)} & \dots & \omega^{-(r-1)(r-1)} \end{pmatrix}}_{F_r^{-1}} \begin{pmatrix} |f(0)\rangle \\ |f(1)\rangle \\ \vdots \\ |f(r-1)\rangle \end{pmatrix}$$

$i = \sqrt{-1}$

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{e=0}^{r-1} \omega^{-se} |f(e)\rangle$$

$$U|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{e=0}^{r-1} \omega^{-se} |f(e+1)\rangle$$

$$= \frac{1}{\sqrt{r}} \sum_{e'=0}^{r-1} \omega^{-s(e'-1)} |f(e')\rangle$$

$$= \omega^s \left[\frac{1}{\sqrt{r}} \sum_{e'=0}^{r-1} \omega^{-se'} |f(e')\rangle \right]$$

$e' = e+1$
 $e = e'-1$

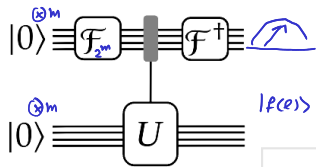
Eigenvalue eqn: $U|u_s\rangle = \omega^s |u_s\rangle$, where $\omega = \exp(i \frac{2\pi}{r})$ is r th root of unity and $s = 0, \dots, r-1$.

$$|0\rangle|0\rangle \xrightarrow{\otimes^m} \left[\frac{1}{\sqrt{2^m}} \sum_{j=0}^{2^m-1} |j\rangle |0\rangle \right]^{\otimes m}$$

$$\rightarrow \frac{1}{\sqrt{2^m}} \sum_{j=0}^{2^m-1} |j\rangle |f(j)\rangle \text{ superposition}$$

$$= \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} \frac{1}{\sqrt{2^m}} \sum_{j=0}^{2^m-1} \omega^{js} |j\rangle |u_s\rangle$$

$$\rightarrow \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} \left| \frac{s 2^m}{r} \right\rangle |u_s\rangle$$



$$|0\rangle^{\otimes m} = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}_{2^m \times 1}$$

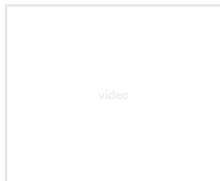
$$|f(e)\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} \omega^{es} |u_s\rangle$$

$$\frac{1}{\sqrt{2^m}} \sum_{j=0}^{2^m-1} \exp\left(i \frac{2\pi}{2^m} \left(\frac{2^m}{r} s\right) j\right) |j\rangle$$

$$\left| \frac{s 2^m}{r} \right\rangle |u_s\rangle$$

$\frac{s 2^m}{r} \rightarrow r$ by continued fractions algorithm

In general, $\frac{s 2^m}{r}$ is not an integer.



Quantum algorithm for order-finding: Example

$$2^{\lceil \log_2(N) \rceil}$$

What are the prime factors of $N = 299$?

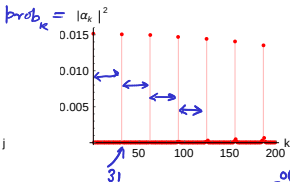
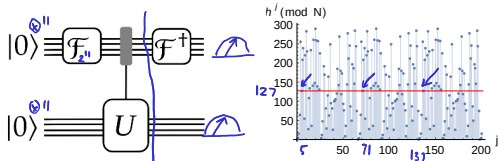
As before let's take $h = 4 < 299 = N$ and $m = 11 \approx \lceil \log_2(N^2) \rceil = 17$

$$|0\rangle^{\otimes m} |0\rangle^{\otimes m} \rightarrow \frac{1}{\sqrt{2^m}} \sum_{j=0}^{2^m-1} |j\rangle |h^j \pmod{N}\rangle = \frac{1}{\sqrt{2^{11}}} (|0\rangle|1\rangle + |1\rangle|4\rangle + |2\rangle|16\rangle + |3\rangle|64\rangle + |4\rangle|256\rangle + |5\rangle|127\rangle + \dots)$$

$4^5 \pmod{299} \equiv 127$
 $4^{11} \pmod{299} \equiv 127$

$$\rightsquigarrow \frac{1}{\sqrt{31}} (|5\rangle + |71\rangle + |137\rangle + |203\rangle + \dots + |1985\rangle) \xrightarrow{F_k^{-1}} \sum_{k=0}^{2^m-1} \alpha_k |\tilde{k}\rangle \rightsquigarrow |31\rangle$$

outcome



$$\frac{31}{2048} = \frac{1}{\frac{2048}{31}} = \frac{1}{66 + \frac{2}{31}} = \frac{1}{66 + \frac{1}{\frac{31}{2}}} = \frac{1}{66 + \frac{1}{15 + \frac{1}{2}}}$$

$31 \times 66 = 2046$

Estimate of $\frac{s}{r} = \frac{31}{2^{11}} = \frac{1}{66 + \frac{1}{15 + \frac{1}{2}}} \approx \frac{1}{66}$ as $|\frac{31}{2^{11}} - \frac{1}{66}| \leq \frac{1}{2^{m+1}}$ holds.

$$CA \sim 2^{O(\ln N^{\frac{1}{3}})}$$

r will be a multiple of 66. Indeed, $4^{66} \pmod{299} \equiv 1$. Takes $O(\log(N)^3)$ steps on QC.

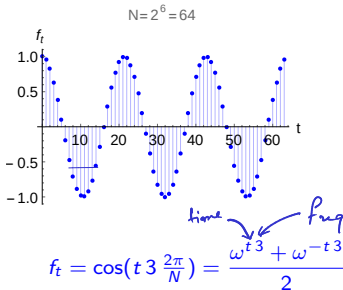
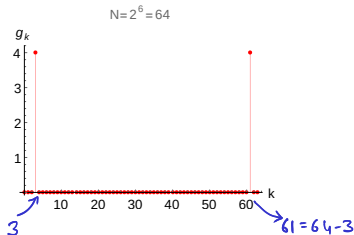
Discrete Fourier transform (DFT)

For n qubits we have $N = 2^n$ and $\omega = \exp(i \frac{2\pi}{N})$ is N th root of unity.

$$\underbrace{\begin{pmatrix} g_0 \\ g_1 \\ \vdots \\ g_{N-1} \end{pmatrix}}_{\mathbf{g}} = \frac{1}{\sqrt{N}} \underbrace{\begin{pmatrix} 1 & 1 & \cdots & 1 \\ 1 & \omega & \cdots & \omega^{N-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \cdots & \omega^{(N-1)(N-1)} \end{pmatrix}}_{\mathbf{F}_N} \underbrace{\begin{pmatrix} f_0 \\ f_1 \\ \vdots \\ f_{N-1} \end{pmatrix}}_{\mathbf{f}} \quad \text{takes } O(N^2) = O(2^{2n}) \text{ steps and finds period.}$$

$N \times N$

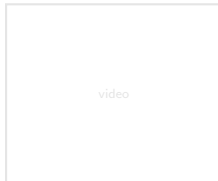
$t = 0, 1, 2, \dots, N-1 = 63$



$$\text{Time Period} = \frac{1}{f} = \frac{1}{3}$$

Uses: real-world data processing

QFT $\sim O(n^2)$



Fast Fourier transform (FFT)

$$t = 257 = 200 + 50 + 7 = 2 \times 10^2 + 5 \times 10^1 + 7 \times 10^0$$

$$= \underbrace{2}_{256} + 1 = 1 \times 2^8 + 0 \times 2^7 + \dots + 0 \times 2^1 + \underbrace{1}_{1} \times 2^0$$

$$= 100000001$$

For n qubits we have $N = 2^n$ and $\omega = \exp(i \frac{2\pi}{N})$ is N th root of unity.

Binary (n -bit) representation: $t = t_1 2^{n-1} + t_2 2^{n-2} + \dots + t_n 2^0 \equiv \underline{t_1 t_2 \dots t_n}$ for $t < 2^n = 9$ $t_i \in \{0, 1\}$

$$k = 0, 1, \dots, N-1$$

DFT: $g_k = \frac{1}{\sqrt{N}} \sum_{t=0}^{N-1} \omega^{kt} f_t$ one summation, which is adding $N = 2^n$ terms $\hookrightarrow O(N^2)$ steps

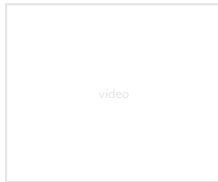
$\omega^{k t_1 2^{n-1}} \quad \omega^{k t_2 2^{n-2}} \quad \dots \quad \omega^{k t_n}$

FFT: $g_k = \frac{1}{\sqrt{N}} \sum_{t_1=0}^1 \dots \sum_{t_n=0}^1 \omega^{k(t_1 2^{n-1} + t_2 2^{n-2} + \dots + t_n 2^0)} f_{t_1 t_2 \dots t_n}$ n summations, each one is adding two terms

$$= \frac{1}{\sqrt{N}} \sum_{t_1=0}^1 \omega^{k t_1 2^{n-1}} \dots \sum_{t_n=0}^1 \omega^{k t_n 2^0} f_{t_1 t_2 \dots t_n}$$
 divide and conquer

FFT takes $O(\underbrace{n 2^n}_N) = O(\underbrace{\log(N) N}_N)$ steps.

J. W. Cooley and J. W. Tukey, Math. Comput. 19, 297 (1965).



Quantum Fourier transform (QFT)

$$k_i = 0, 1$$

For n qubits we have $N = 2^n$ and $\omega = \exp(i \frac{2\pi}{N})$ is N th root of unity.

Binary (n -qubit) representation: $k = k_1 2^{n-1} + k_2 2^{n-2} + \dots + k_n 2^0 \Rightarrow |k\rangle \equiv |k_1 k_2 \dots k_n\rangle$

$$F^\dagger F = I = F F^\dagger$$

$$\begin{aligned} \text{QFT: } \mathcal{F}|t\rangle &= \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega^{tk} |k\rangle \\ &= \frac{1}{\sqrt{N}} \sum_{k_1=0}^1 \dots \sum_{k_n=0}^1 \omega^{t(k_1 2^{n-1} + k_2 2^{n-2} + \dots + k_n 2^0)} |k_1 k_2 \dots k_n\rangle \\ &= \frac{1}{\sqrt{N}} \left[\sum_{k_1=0}^1 \omega^{t k_1 2^{n-1}} |k_1\rangle \right] \otimes \dots \otimes \left[\sum_{k_n=0}^1 \omega^{t k_n 2^0} |k_n\rangle \right] \end{aligned}$$

$|k_1\rangle \otimes |k_2\rangle \otimes \dots \otimes |k_n\rangle$

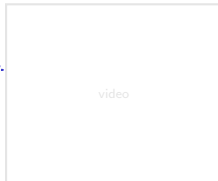
Power of superposition
(a.m)
(a.c)

$$\begin{aligned} \sum_{k=0}^{N-1} g_k |k\rangle &= \mathcal{F} \left[\sum_{t=0}^{N-1} f_t |t\rangle \right] \\ &= \sum_{t=0}^{N-1} \underbrace{\left[\frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega^{tk} f_t |k\rangle \right]}_{g_k} \end{aligned}$$

Exponential data compression: storing $N = 2^n$ complex numbers $\{f_t\}$ in n qubits.

Exponentially fast: QFT takes $O(n^2) = O(\log(N)^2)$ computation steps.

Using Superposition.
|4>



Quantum Fourier transform (QFT)

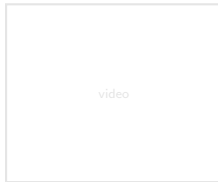
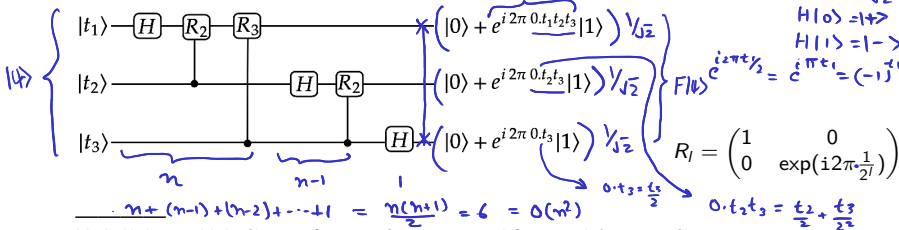
$$N = 2^n, \quad \omega^{t 2^{(n-l)}} = \exp(i \underbrace{\frac{2\pi}{N} t 2^{(n-l)}}_{2\pi \frac{t}{2^{n-(n-l)}}}) = \exp(i 2\pi \underbrace{\frac{t}{2^l}}_{\text{Integer}}) \text{ and } \frac{t}{2^l} = \underbrace{t_1 2^{n-1-l} + \dots + t_{n-l} 2^0}_{\text{Integer}} + \underbrace{t_{n-l+1} \frac{1}{2} + \dots + t_n \frac{1}{2^l}}_{\text{last } l \text{ bits of } t}$$

$$\mathcal{F}|t\rangle = \frac{1}{\sqrt{N}} \left[\sum_{k_1=0}^1 \omega^{t k_1 2^{n-1}} |k_1\rangle \right] \otimes \dots \otimes \left[\sum_{k_n=0}^1 \omega^{t k_n 2^0} |k_n\rangle \right] = \frac{1}{\sqrt{2}} [|0\rangle + \omega^{t 2^{n-1}} |1\rangle] \otimes \dots \otimes \frac{1}{\sqrt{2}} [|0\rangle + \omega^{t 2^0} |1\rangle]$$

$$\mathcal{F}|t_1 t_2 \dots t_n\rangle = \frac{1}{\sqrt{2}} [|0\rangle + e^{i 2\pi 0.t_n} |1\rangle] \otimes \dots \otimes \frac{1}{\sqrt{2}} [|0\rangle + e^{i 2\pi 0.t_1 \dots t_n} |1\rangle]$$

$$0.257 = 0.2 + 0.05 + 0.007 \\ = \frac{2}{10} + \frac{5}{10^2} + \frac{7}{10^3}$$

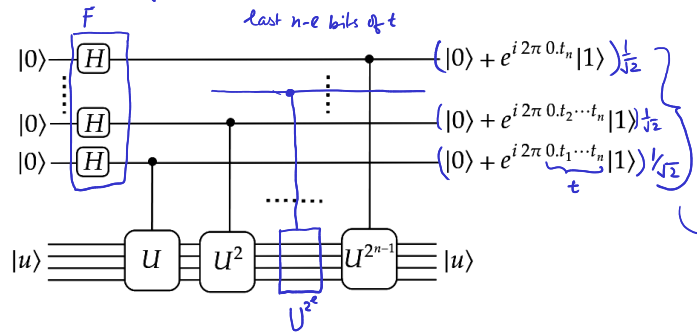
$n=3$



Q π_{AI} $t \in \{0,1\}$ Unknown phase (eigenvalue) estimation

$$t \approx 0.t_1 t_2 \dots t_n = \frac{t_1}{2} + \frac{t_2}{2^2} + \dots + \frac{t_n}{2^n} \in [0, 1)$$

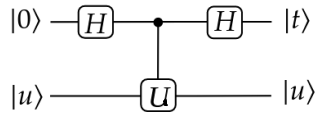
$$t 2^l \approx \underbrace{t_1 2^{l-1} + \dots + t_l}_{\text{Integer}} + \underbrace{\frac{t_{l+1}}{2} + \dots + \frac{t_n}{2^{n-l}}}_{0.t_{l+1} \dots t_n}$$



$$U^{2^e} |u\rangle = e^{i 2\pi t \frac{2^e}{2}} |u\rangle$$

Phase kickback: $e^{i\theta}$ $\theta \in \mathbb{R}$
 $i = \sqrt{-1}$

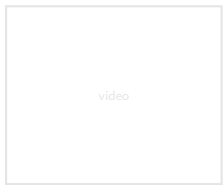
$$U|u\rangle = e^{i 2\pi t} |u\rangle$$



$$|0\rangle|u\rangle \rightarrow \frac{1}{\sqrt{2}} [|0\rangle|u\rangle + |1\rangle|u\rangle] \rightarrow \frac{1}{\sqrt{2}} [|0\rangle + e^{i 2\pi t} |1\rangle] |u\rangle$$

$$|0\rangle|u\rangle \mapsto \left(\frac{1+e^{i 2\pi t}}{2} |0\rangle + \frac{1-e^{i 2\pi t}}{2} |1\rangle \right) |u\rangle$$

$F^{-1} \rightarrow |t_1 t_2 \dots t_n\rangle$



Uses: Shor factoring algorithm and Quantum HHL algorithm for system of linear eqs.

Amplitude estimation: Grover + Shor algorithm

$$j \rightarrow |j\rangle$$

$$|\psi\rangle = \sum_{j \in X} \alpha_j |j\rangle = \underbrace{\sin \theta \sum_{j \in X_{\text{good}}} \alpha_j |j\rangle}_{|\psi_{\text{good}}\rangle} + \underbrace{\sum_{j \in X_{\text{bad}}} \alpha_j |j\rangle \frac{1}{\cos \theta}}_{|\psi_{\text{bad}}\rangle} \cos \theta$$

For given

$$X = \{0, 1, \dots, N-1\} = X_{\text{good}} \cup X_{\text{bad}}, \text{ where } X_{\text{good}} \cap X_{\text{bad}} = \emptyset$$

$$A|0\rangle = \sum_{j \in X} \alpha_j |j\rangle = \sin \theta |\psi_{\text{good}}\rangle + \cos \theta |\psi_{\text{bad}}\rangle =: |\psi\rangle, \text{ where } (\sin \theta)^2 = \sum_{j \in X_{\text{good}}} |\alpha_j|^2 \text{ and } (\cos \theta)^2 = \sum_{j \in X_{\text{bad}}} |\alpha_j|^2$$

$$V|\psi_{\text{good}}\rangle = -|\psi_{\text{good}}\rangle \text{ and } V|\psi_{\text{bad}}\rangle = |\psi_{\text{bad}}\rangle \quad \frac{|\psi_{\text{good}}\rangle + |\psi_{\text{bad}}\rangle}{\sqrt{2}} = |\psi_{\text{avg}}\rangle, \quad (-i)(\frac{|\psi_{\text{good}}\rangle - |\psi_{\text{bad}}\rangle}{\sqrt{2}}) = |\psi_{\text{diff}}\rangle$$

The task is to estimate the amplitude $\sin \theta \in [0, 1]$. Amplitude "sin θ " estimation $\leftrightarrow$ phase " θ " estimation

$$U = W V, \text{ where } W = 2|\psi\rangle\langle\psi| - I,$$

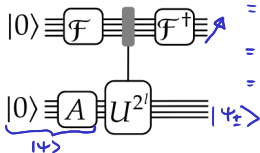
$$U \begin{pmatrix} |\psi_{\text{good}}\rangle \\ |\psi_{\text{bad}}\rangle \end{pmatrix} = \begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix} \begin{pmatrix} |\psi_{\text{good}}\rangle \\ |\psi_{\text{bad}}\rangle \end{pmatrix}$$

$$U|\psi_{\pm}\rangle = e^{\pm i2\theta} |\psi_{\pm}\rangle$$

$$|\psi_{\pm}\rangle = \frac{\pm i |\psi_{\text{good}}\rangle + |\psi_{\text{bad}}\rangle}{\sqrt{2}}$$

$$|\psi\rangle = \frac{e^{-i\theta} |\psi_{+}\rangle + e^{i\theta} |\psi_{-}\rangle}{\sqrt{2}}$$

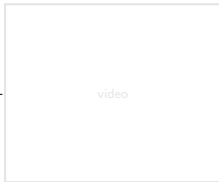
$$U \begin{pmatrix} i \\ 1 \end{pmatrix} = \begin{pmatrix} i \cos 2\theta - \sin 2\theta \\ i \sin 2\theta + \cos 2\theta \end{pmatrix} = \begin{pmatrix} i e^{i2\theta} \\ e^{i2\theta} \end{pmatrix} = e^{i2\theta} \begin{pmatrix} i \\ 1 \end{pmatrix}$$



$$|0\rangle|\psi\rangle \rightarrow \frac{e^{-i\theta} |\frac{\theta}{\pi}\rangle |\psi_{+}\rangle + e^{i\theta} |-\frac{\theta}{\pi}\rangle |\psi_{-}\rangle}{\sqrt{2}}$$

$$e^{-i2\theta} = e^{-i2\pi \frac{\theta}{\pi}}$$

$$\begin{aligned} U|\psi_{\text{good}}\rangle &= W V |\psi_{\text{good}}\rangle = -W |\psi_{\text{good}}\rangle \\ &= -[2|\psi_{\text{avg}}\rangle\langle\psi_{\text{good}}| - |\psi_{\text{good}}\rangle\langle\psi_{\text{good}}| - |\psi_{\text{bad}}\rangle\langle\psi_{\text{bad}}|] |\psi_{\text{good}}\rangle \\ &= -[2\sin \theta (\sin \theta |\psi_{\text{good}}\rangle + \cos \theta |\psi_{\text{bad}}\rangle) - |\psi_{\text{good}}\rangle] \\ &= (1 - 2\sin^2 \theta) |\psi_{\text{good}}\rangle - 2\sin \theta \cos \theta |\psi_{\text{bad}}\rangle \end{aligned}$$



P. Kaye, R. Laflamme, and Michele Mosca, An Introduction to Quantum Computing, Chapt 8.

P. Rebentrost, B. Gupt, and T. R. Bromley, Phys. Rev. A **98**, 022321 (2018).

What are the prime factors of $N = 91$?

Step 1: find $x \pmod{N} \not\equiv \pm 1$ but $x^2 \pmod{N} \equiv 1$

$$(x - 1)(x + 1) = k \times N$$

For $x = -1, \dots, 100$,

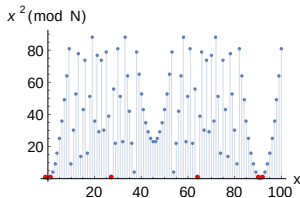
$\{x, x^2 \pmod{N}\} = \{\textcolor{red}{-1}, \textcolor{red}{1}\}, \{\textcolor{yellow}{0}, \textcolor{yellow}{0}\}, \{\textcolor{red}{1}, \textcolor{red}{1}\}, \{2, 4\}, \{3, 9\}, \{4, 16\}, \{5, 25\}, \{6, 36\}, \{7, 49\}, \{8, 64\}, \{9, 81\}, \{10, 9\}, \{11, 30\}, \{12, 53\}, \{13, 78\}, \{14, 14\}, \{15, 43\}, \{16, 74\}, \{17, 16\}, \{18, 51\}, \{19, 88\}, \{20, 36\}, \{21, 77\}, \{22, 29\}, \{23, 74\}, \{24, 30\}, \{25, 79\}, \{26, 39\}, \{\textcolor{red}{27}, \textcolor{red}{1}\}, \{28, 56\}, \{29, 22\}, \{30, 81\}, \{31, 51\}, \{32, 23\}, \{33, 88\}, \{34, 64\}, \{35, 42\}, \{36, 22\}, \{37, 4\}, \{38, 79\}, \{39, 65\}, \{40, 53\}, \{41, 43\}, \{42, 35\}, \{43, 29\}, \{44, 25\}, \{45, 23\}, \{46, 23\}, \{47, 25\}, \{48, 29\}, \{49, 35\}, \{50, 43\}, \{51, 53\}, \{52, 65\}, \{53, 79\}, \{54, 4\}, \{55, 22\}, \{56, 42\}, \{57, 64\}, \{58, 88\}, \{59, 23\}, \{60, 51\}, \{61, 81\}, \{62, 22\}, \{63, 56\}, \{\textcolor{red}{64}, \textcolor{red}{1}\}, \{65, 39\}, \{66, 79\}, \{67, 30\}, \{68, 74\}, \{69, 29\}, \{70, 77\}, \{71, 36\}, \{72, 88\}, \{73, 51\}, \{74, 16\}, \{75, 74\}, \{76, 43\}, \{77, 14\}, \{78, 78\}, \{79, 53\}, \{80, 30\}, \{81, 9\}, \{82, 81\}, \{83, 64\}, \{84, 49\}, \{85, 36\}, \{86, 25\}, \{87, 16\}, \{88, 9\}, \{89, 4\}, \{\textcolor{red}{90}, \textcolor{red}{1}\}, \{\textcolor{yellow}{91}, \textcolor{yellow}{0}\}, \{\textcolor{red}{92}, \textcolor{red}{1}\}, \{93, 4\}, \{94, 9\}, \{95, 16\}, \{96, 25\}, \{97, 36\}, \{98, 49\}, \{99, 64\}, \{100, 81\}\}$

$x = 27$

Step 2: $p = \gcd(x - 1, N) = (26, 91) = 13$

$q = \gcd(x + 1, N) = (28, 91) = 7$

$$\frac{x-1}{N} = \frac{2}{7}, \quad \frac{x+1}{N} = \frac{4}{13}$$



video

- The function $f(j) := h^j \pmod{N}$ has only r distinct values $f_\tau = f(\tau)$ for $\tau = 0, \dots, r-1$.
- From 0 to $2^m - 1$, suppose only $j_\tau, j_\tau + r, j_\tau + 2r, \dots, j_\tau + (\xi_\tau - 1)r$ (in ascending order) give the same value f_τ , where $\xi_\tau = \lfloor \frac{2^m - 1 - j_\tau}{r} \rfloor + 1$ due to $j_\tau + (\xi_\tau - 1)r \leq 2^m - 1$.

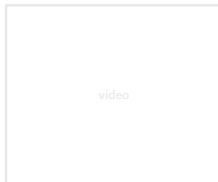
$$\xi_\tau = 31$$

$$\frac{1}{\sqrt{2^m}} \sum_{j=0}^{2^m-1} |j\rangle |f(j)\rangle = \frac{1}{\sqrt{2^m}} \sum_{\tau=0}^{r-1} \sum_{l=0}^{\xi_\tau-1} |j_\tau + lr\rangle |f_\tau\rangle \quad \text{--- meas on 2nd reg} \longrightarrow \left[\frac{1}{\sqrt{\xi_\tau}} \sum_{l=0}^{\xi_\tau-1} |j_\tau + lr\rangle \right] |f_\tau\rangle$$

$$\frac{1}{\sqrt{\xi_\tau}} \sum_{l=0}^{\xi_\tau-1} \mathcal{F}^{-1} |j_\tau + lr\rangle = \frac{1}{\sqrt{\xi_\tau 2^m}} \sum_{k=0}^{2^m-1} \sum_{l=0}^{\xi_\tau-1} \exp\left(-i \frac{2\pi}{2^m} (j_\tau + lr)k\right) |\tilde{k}\rangle \quad \text{--- meas on 1st reg} \longrightarrow |\tilde{k}\rangle$$

Probability of getting $|f_\tau\rangle$ and then $|\tilde{k}\rangle = \frac{1}{\xi_\tau 2^m} \left| \sum_{l=0}^{\xi_\tau-1} \exp\left(-i \frac{2\pi}{2^m} (j_\tau + lr)k\right) \right|^2$

$$|\alpha_k|^2 = \frac{1}{\xi_\tau 2^m} \left| \sum_{l=0}^{\xi_\tau-1} \exp\left(-i \frac{2\pi}{2^m} l r k\right) \right|^2$$



For n qubits we have $N = 2^n$ and $\omega = \exp(i \frac{2\pi}{N})$ is N th root of unity.

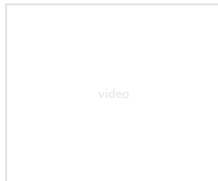
DFT: $g_k = \frac{1}{\sqrt{N}} \sum_{t=0}^{N-1} \omega^{kt} f_t = [F_N \mathbf{f}]_k$

$$= \frac{1}{\sqrt{N}} \left[\sum_{j=0}^{\frac{N}{2}-1} \omega^{k(2j)} f_{2j} + \sum_{j=0}^{\frac{N}{2}-1} \omega^{k(2j+1)} f_{2j+1} \right] \quad \text{divide and conquer}$$

$$= \frac{1}{\sqrt{N}} \left[\sum_{j=0}^{\tilde{N}-1} \tilde{\omega}^{kj} f_{2j} + \omega^k \sum_{j=0}^{\tilde{N}-1} \tilde{\omega}^{kj} f_{2j+1} \right] \quad \text{where } \tilde{N} = \frac{N}{2} = 2^{n-1} \text{ and } \tilde{\omega} = \omega^2 \text{ in } \tilde{N}\text{th root of unity}$$

FFT: $= \underbrace{\frac{\sqrt{\tilde{N}}}{\sqrt{N}}}_{\frac{1}{\sqrt{2}}} \left([\tilde{F}_{\frac{N}{2}} \mathbf{f}^{\text{even}}]_k + \omega^k [\tilde{F}_{\frac{N}{2}} \mathbf{f}^{\text{odd}}]_k \right)$ takes $O(n 2^n) = O(\log(N) N)$ steps

J. W. Cooley and J. W. Tukey, Math. Comput. 19, 297 (1965).



Q π _{AI} Modular (remainder) arithmetic and multiplicative group of co-primes

$$x \equiv y \pmod{N}$$

- $y \equiv \underbrace{x \pmod{N}}_{\text{computing remainder}} \leftrightarrow \underbrace{x}_{\text{dividend}} = \underbrace{y}_{\text{remainder}} + \underbrace{k}_{\text{quotient}} \times \underbrace{N}_{\text{divisor}} \leftrightarrow x \text{ and } y \text{ are equal up to a multiple of } N.$

$$13\text{hr} \equiv 1\text{pm}$$

$$N = 12 \quad \sqrt{13 = x} \quad 1 = k$$

$$\frac{12}{1} = y$$

$$13 \equiv 1 \pmod{12}$$

$$\mathbb{Z}_{12} = \{0, 1, 2, \dots, 11\} \text{ additive gp}$$

$$6 + 7 = 13 \equiv 1 \pmod{12}$$

$$\textcircled{3} (a+b)+c = a+(b+c)$$

$$\textcircled{1} 6 + 0 = 6$$

$$\textcircled{2} 6 + 6 = 12 \equiv 0 \pmod{12} \quad \text{Prime: } 2, 3, 5, 7, 11, 13 \dots$$

- E and M are called co-primes, if they have no common factors, that is $\gcd(E, M) = 1$.

$$6 = 2 \times 3 \quad 55 = 5 \times 11 \quad \gcd(6, 55) = 1$$

$$3 = 1 \times 3$$

$$5 = 1 \times 5$$

- $\mathbb{Z}_M^* = \{E \mid \underbrace{0 \leq E \leq M-1}_{\mathbb{Z}_M} \text{ and } \gcd(E, M) = 1\} = \text{the multiplicative group of integers modulo-}M.$

$$\gcd(3, 5) = 1$$

$$\mathbb{Z}_{264}^* = \{1, 5, 7, 13, 17, 19, 23, 25, 29, 31, 35, 37, 41, 43, 47, 49, 53, 59, 61, 65, 67, 71, 73, 79, 83, 85, 89, 91, 95, 97, 101, 103, 107, 109, 113, 115, 119, 125, 127, 131, 133, 137, 139, 145, 149, 151, 155, 157, 161, 163, 167, 169, 173, 175, 179, 181, 185, 191, 193, 197, 199, 203, 205, 211, 215, 217, 221, 223, 227, 229, 233, 235, 239, 241, 245, 247, 251, 257, 259, 263\}$$

$$\gcd(2, 264) = 2$$

$$\gcd(3, 264) = 3$$

$$5 \times 7 = 35, \quad 5 \times 53 = 265 = 1 + 264, \quad 19 \times 31 = 589 = 61 + 2 \times 264$$

$$\equiv 35 \pmod{264}$$

$$\equiv 1 \pmod{264}$$

$$\equiv 61 \pmod{264}$$

$$5 \times 1 = 5$$

$\gcd(0, M) = M$, " $\equiv$ " means congruent

N. D. Mermin, *Quantum Computer Science: An Introduction*, Chapt 3.

M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Appendix 4 and Chapt 5.

RSA (Rivest–Shamir–Adleman) encryption

Alice -----> Bob

Step 1: Bob "randomly" chooses two ^{odd} distinct prime numbers p and q , say, each of 1000 digits.

Step 2: Bob computes $N = pq$ and $M = (p-1)(q-1)$. Without knowing p and q , M cannot be calculated.

Step 3: Bob chooses an **encoding** integer $E \in \mathbb{Z}_M^*$.

Step 4: Bob sends N and E to Alice through a public channel.



Step 5: Alice encodes her message $a < N$ as $b \equiv a^E \pmod{N}$ and then sends b to Bob through a public channel.

Step 6: Bob computes **decoding** number (multiplicative inverse) D such that $1 \equiv DE \pmod{M}$, and then decode the original message $a \equiv b^D \pmod{N}$. Without knowing M , D cannot be calculated.

best classical algo $\sim 2^{O(n^{1/3})}$ steps

Shor algo on QC $\sim O(n^3)$ steps

$$n = \lceil \log_2 N \rceil = \text{min number of bits needed to rep. } N$$

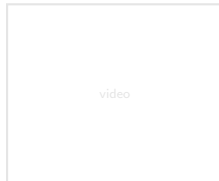
$$\lceil 3.2 \rceil = 4$$

$$N = 7 \equiv 111$$

Finding p and q from N is called prime factorization, which is practical impossibility for a "classical" computer for a 2000-digit number.

$$p \sim 10^{1000}$$

$$N \sim 10^{2000}$$



RSA (Rivest–Shamir–Adleman) encryption: Example

Alice -----> Bob

Step 1: Bob randomly chooses two distinct ^{odd} prime numbers $p = 13$ and $q = 23$.

Primality test
 Yes
 No

Step 2: Bob computes $N = pq = 299$ and $M = (p - 1)(q - 1) = 264$.
 12×22

Step 3: Bob chooses an **encoding** integer $E = 5 \in \mathbb{Z}_M^* =$
 $\{1, 5, 7, 13, 17, 19, 23, 25, 29, 31, 35, 37, 41, 43, 47, 49, 53, 59, 61, 65, 67, 71, 73, 79, 83, 85, 89, 91, 95, 97, 101, 103, 107, 109, 113, 115, 119, 125, 127, 131, 133, 137, 139, 145, 149, 151, 155, 157, 161, 163, 167, 169, 173, 175, 179, 181, 185, 191, 193, 197, 199, 203, 205, 211, 215, 217, 221, 223, 227, 229, 233, 235, 239, 241, 245, 247, 251, 257, 259, 263\}$

Step 4: Bob sends $N = 299$ and $E = 5$ to Alice through a public channel.

$$20^5 = 3200000 = 10702 \times 299 + 102$$

Step 5: Alice encodes her message $a = 20$ as $b = a^E \pmod{N} = 102$ and then sends b to Bob through a public channel.

Step 6: Bob computes **decoding** number (multiplicative inverse) $D = 53$ such that $1 \equiv DE \pmod{M}$, and then decode the original message $b^D \pmod{N} = 20$.

$$299 \overline{) 102^{53}}$$

